



I.E.S. Laguna de Joatzel

TRABAJO DE FIN DE CICLO

Shinny Hunt

**CICLO FORMATIVO DE GRADO SUPERIOR
DESARROLLO DE APLICACIONES WEB**

17/05/2026

Delia Méndez Navarro

Tutor: David García

Indice

Introducción.....	4
Presentación.....	4
Contexto.....	4
Objetivos.....	4
Desarrollo.....	6
Temporalización y ciclo de vida.....	6
Trabajo de análisis.....	7
Descripción general.....	7
Usuario como comprador.....	7
Búsqueda y exploración de cartas.....	7
Ficha de carta y listings.....	7
Carrito de compra y pasarela de pago.....	8
Wishlist.....	8
Colección personal.....	8
Usuario como vendedor.....	8
Publicación de listings.....	8
Gestión de ventas.....	8
Saldo acumulado.....	9
Gestión de cuenta y perfil.....	9
Comunicación entre usuarios.....	9
Búsqueda.....	9
Casos de uso.....	10
Registro e inicio de sesión.....	10
Búsqueda y exploración de cartas.....	10
Compra de cartas.....	10
Publicación y gestión de listings.....	10
Gestión de la colección personal.....	10
Wishlist.....	11
Mensajería privada.....	11
Gestión del perfil.....	11
Perfil público de otros usuarios.....	11
Trabajo de diseño.....	12
Diagrama modelo entidad relación.....	12
Diseño de módulos/ficheros de código.....	13
Diagrama de clases.....	14
Diagrama de navegación.....	15
Trabajo de implementación.....	16
Introducción.....	16
Tecnologías y Herramientas.....	16
Backend.....	16
Librerías y APIs.....	16
Frontend.....	17
Herramientas de Desarrollo.....	17
Desarrollo.....	17
Backend.....	17
Modelos implementados.....	20
Validaciones implementadas.....	20
Frontend.....	21
Características implementadas.....	21
Validación frontend.....	22

Justificación.....	22
Trabajo de pruebas.....	23
Trabajo de implantación.....	23
Entorno de desarrollo y Servidor local.....	23
Configuración de servicios externos.....	24
Arquitectura de despliegue y rutas.....	24
Proceso de puesta en producción.....	24
Guía de instalación y ejecución local.....	25
Producto.....	26
Descripción externa.....	26
Página de inicio.....	26
Nuevo registro.....	26
Login.....	27
Usuario con sesión iniciada.....	27
Detalle de carta.....	28
Formulario de venta.....	29
Buscador de carta.....	29
Buscador de usuario.....	30
Perfil de usuario ajeno.....	30
Colección.....	31
Lista de deseos.....	31
Mensajes.....	32
Carrito.....	32
Perfil de usuario.....	33
Venta.....	34
Infraestructura.....	34
Entorno de ejecución local.....	34
Gestión de la Seguridad y Configuración.....	35
Integración de Servicios en la Nube (APIs).....	35
Organización de Rutas y Navegación.....	35
Conclusión.....	36
Resumen del proyecto.....	36
Lecciones aprendidas.....	36
Posible evolución futura.....	36
Bibliografía.....	37

Introducción.

Presentación.

Nos encontramos ante una aplicación web dedicada a la gestión integral y el comercio de las cartas coleccionables. Esta aplicación permite a los usuarios registrar su propia colección, gestionar su lista de deseos y participar en un mercado de segunda mano especializado donde pueden comprar y poner a la venta cartas coleccionables TCG. La plataforma destaca por permitir una descripción detallada del estado físico de la carta, con una descripción si es necesario ponerla y cuenta también con un chat interno.

Contexto

En el presente proyecto se ha desarrollado una aplicación web utilizando un enfoque full-stack basado en tecnologías nativas y de amplio uso en el entorno servidor. El backend se ha implementado en PHP, encargado de la lógica de negocio, gestión de sesiones de usuario y conexión con la base de datos. Para el frontend, se ha utilizado una combinación de HTML5 para la estructura, CSS3 para el diseño visual y JavaScript para dotar de interactividad a la interfaz, como la validación de formularios y gestión dinámica de los inventarios de cartas. El sistema se ha diseñado para funcionar en un entorno servidor local (como Apache), lo que permite validar la integración de todos los módulos y la persistencia de datos de forma eficiente.

Objetivos

- Desarrollar una aplicación web completa.
 - Crear un backend funcional y estructurado utilizando PHP para el procesamiento de datos.
 - Diseñar una interfaz de usuario clara y atractiva utilizando HTML5, CSS3 y JS.
- Garantizar la transparencia en el mercado.
 - Implementar un modelo de datos que obligue a detallar la condición física de la carta para evitar ambigüedades en las transacciones.

Delia Méndez Navarro

- Establecer un sistema de mensajería interna que asegure la comunicación entre coleccionistas.
- Mejorar la experiencia del coleccionista.
 - Permitir la organización de posesiones y lista de deseos de forma centralizada.
 - Facilitar la búsqueda y el intercambio de cartas mediante un entorno estructurado.
- Asegurar un desarrollo estructurado.
 - Validar el proyecto mediante un despliegue local que garantice que el sistema responde correctamente a los requisitos de usabilidad y seguridad.

Desarrollo.

Temporalización y ciclo de vida

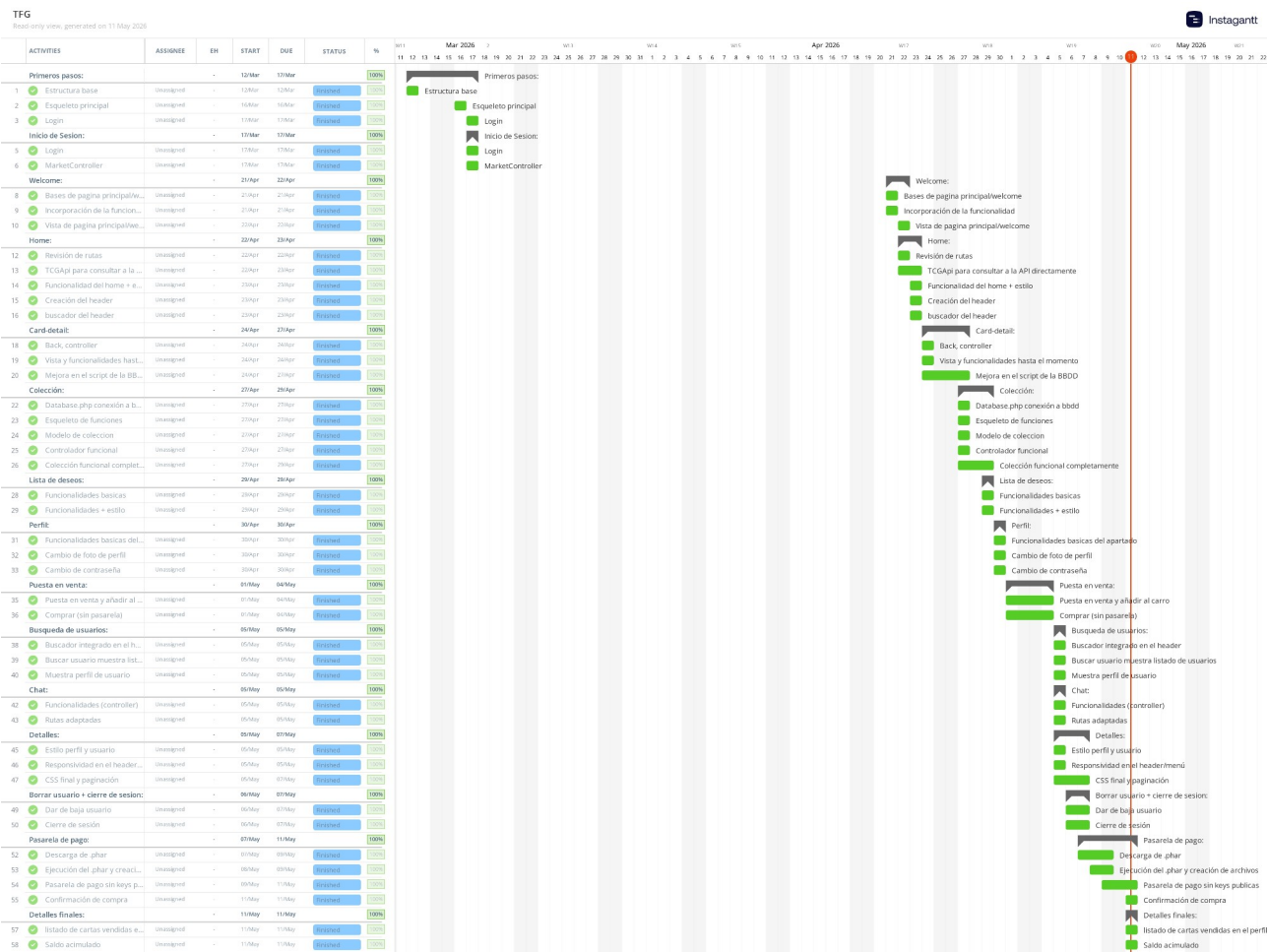
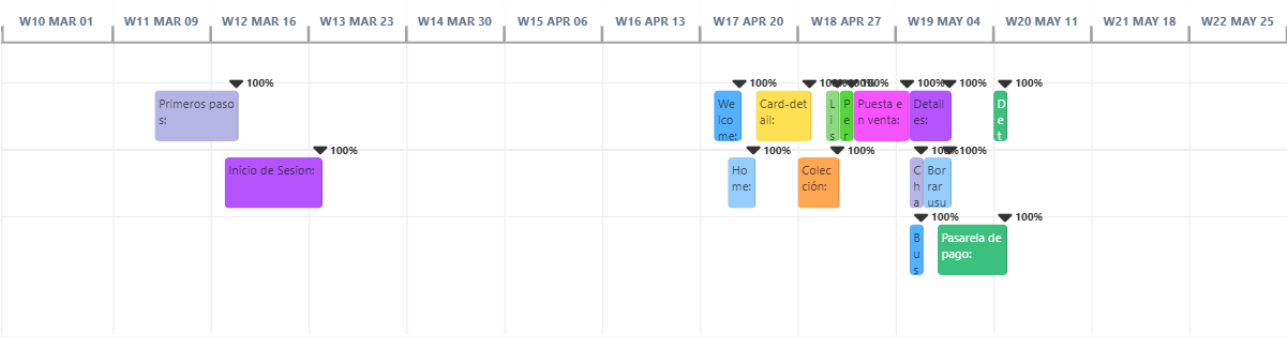


Figura 1: Diagrama de Gantt

El desarrollo del proyecto se ha articulado siguiendo el ciclo de vida iterativo e incremental, tal y como se detalla en el diagrama de Gantt. Esta planificación ha permitido un progreso estructurado desde la primera creación de las bases en marzo hasta la implementación de críticos en mayo, facilitando así la detección temprana de riesgos y la integración continua de funcionalidades.

El trabajo en su totalidad ha sido desarrollado por Delia Méndez Navarro.

Trabajo de análisis

Descripción general.

ShinnyHunt es una aplicación web de tipo *marketplace* orientada al intercambio de cartas coleccionables entre usuarios particulares. La plataforma permite a cualquier usuario registrado tanto comprar cartas publicadas por otros usuarios como poner en venta las suyas propias, convirtiéndola en un mercado horizontal sin figura de administración central.

Usuario como comprador

La vertiente de compra de la plataforma ofrece al usuario todas las herramientas necesarias para localizar, evaluar y adquirir cartas coleccionables de otros usuarios.

Búsqueda y exploración de cartas.

Los usuarios pueden buscar cartas coleccionables a través del buscador integrado. La búsqueda devuelve los resultados con información relevante de cada carta (nombre e imagen), permitiendo explorar el catálogo disponible en el *marketplace*.

Ficha de carta y listings.

Cada carta dispone de una ficha detallada donde se muestran sus características técnicas (rareza, tipo, set, ataques, debilidades) junto con el listado de vendedores activos que ofrecen esa carta ordenados por precio. Desde esta vista el usuario puede añadir cualquier listing al carrito de compra.

Carrito de compra y pasarela de pago.

La aplicación dispone de un carrito persistente donde el usuario puede acumular cartas de distintos vendedores antes de proceder al pago. El proceso de pago se realiza a través de Stripe, garantizando una transacción segura. Tras el pago exitoso, el sistema actualiza automáticamente el stock de cada listing involucrado y acredita el importe correspondiente al saldo virtual de cada vendedor.

Wishlist.

El usuario puede marcar cartas como deseadas en su wishlist personal. Esta lista le permite llevar un seguimiento de las cartas que le interesan para adquirirlas en el futuro.

Colección personal.

Independientemente de las transacciones comerciales, el usuario puede gestionar su colección personal añadiendo y eliminando cartas que ya posee. Esta funcionalidad es puramente organizativa y permite al usuario llevar un inventario de su colección.

Usuario como vendedor

Cualquier usuario registrado puede adoptar el rol de vendedor, publicando listings de cartas que desee poner a la venta.

Publicación de listings.

Desde la ficha de cualquier carta, el usuario puede crear un listing especificando el precio, la cantidad disponible, la condición de la carta (Near Mint, Excellent, Good, Played) y una descripción opcional. Una vez publicado el listing aparece en la ficha de la carta para el resto del usuarios.

Gestión de ventas.

El apartado de “Mis Ventas” ofrece al usuario una vista consolidada de todos sus listings, con información del stock, precio, fecha de publicación y estado. Desde esta sección el vendedor puede cancelar listings activos que no desee mantener.

Saldo acumulado.

Cada vez que uno de los listings es comprado, el sistema acredita automáticamente el importe de la venta en el saldo virtual del vendedor. Este saldo es visible en el perfil de usuario y puede ser consultado en cualquier momento.

Gestión de cuenta y perfil.

La plataforma gestiona el ciclo de vida completo del usuario, desde el registro hasta la baja.

- Registro con nombre de usuario, email, contraseña, numero de teléfono y dirección.
- Inicio y cierre de sesión con autenticación segura.
- Edición de perfil: nombre de usuario, email, teléfono, biografía, contraseña y dirección.
- Subida y eliminación de avatar personalizado.
- Baja de cuenta permanente con confirmación explícita.

Comunicación entre usuarios

ShinnyHunt incorpora un sistema de mensajería privada que permite a los usuarios comunicarse directamente entre sí. Esto resulta especialmente útil para coordinar como detalles de envío o negociación de condiciones de la venta fuera del proceso de pago, automatizado. Los usuarios pueden ver el historial de conversaciones ordenado cronológicamente y acceder al perfil publico del interlocutor.

Búsqueda

El buscador de la plataforma cubre dos entidades diferenciadas:

- Búsqueda de cartas. Permite localizar cualquier carta del catálogo por nombre u otros parámetros, mostrando resultados con imagen, identificador y disponibilidad en el mercado.
- Búsqueda por usuarios. Permite encontrar el perfil público de otros usuarios por nombre de usuario, facilitando el acceso a su catálogo de listings activos.

Casos de uso.

Registro e inicio de sesión

Como usuario nuevo quiero poder registrarme en la plataforma proporcionando mis datos básicos, para acceder a todas las funcionalidades del marketplace. Una vez registrado, quiero poder iniciar sesión de forma segura y cerrar sesión cuando lo desee.

Búsqueda y exploración de cartas.

Como usuario quiero buscar cartas coleccionables por nombre para encontrar rápidamente las que me interesan. Al acceder a la ficha de una carta, quiero ver sus características técnicas completas y el listado de vendedores que la ofrecen con sus respectivos precios y condiciones, para poder tomar una decisión de compra informada.

Compra de cartas.

Como comprador, quiero añadir cartas de distintos vendedores a mi carrito para realizar una compra conjunta. Necesito que el sistema controle el stock disponible para evitar compras no válidas. Al proceder al pago quiero hacerlo de forma segura a través de una pasarela de pago, y recibir confirmación inmediata del resultado de la transacción.

Publicación y gestión de listings

Como vendedor, quiero publicar anuncios de venta de mis cartas especificando precio, cantidad, condición y descripción opcional. Quiero poder ver todos mis listings activos y vendidos en un único apartado, y cancelar aquellos que ya no desee mantener activos.

Gestión de la colección personal

Como coleccionista, quiero mantener un registro de las cartas que poseo en mi colección personal, pudiendo añadir y eliminar cartas según mi inventario real. Esta funcionalidad es independiente del marketplace y me sirve como herramienta de organización personal.

Wishlist

Como usuario, quiero marcar cartas como deseadas en mi wishlist para llevar un seguimiento de las que me gustaría adquirir en el futuro. Quiero poder añadir y eliminar cartas de esta lista en cualquier momento.

Mensajería privada

Como usuario, quiero poder enviar mensajes privados a otros usuarios de la plataforma para coordinar detalles de envío o resolver dudas sobre un listing. Quiero tener acceso a un historial de todas mis conversaciones activas, ordenado por fecha del último mensaje.

Gestión del perfil

Como usuario, quiero poder editar mis datos personales (nombre de usuario, email, teléfono, biografía), cambiar mi contraseña y personalizar mi perfil con un avatar. También quiero poder consultar mi saldo acumulado por ventas y, si decido abandonar la plataforma, dar de baja mi cuenta de forma permanente.

Perfil público de otros usuarios

Como usuario, quiero poder visitar el perfil público de otros vendedores para ver su información básica y el listado de cartas que tienen actualmente en venta, facilitando la exploración del catálogo por vendedor e iniciando una conversación directa si lo necesito.

Trabajo de diseño

Diagrama modelo entidad relación.

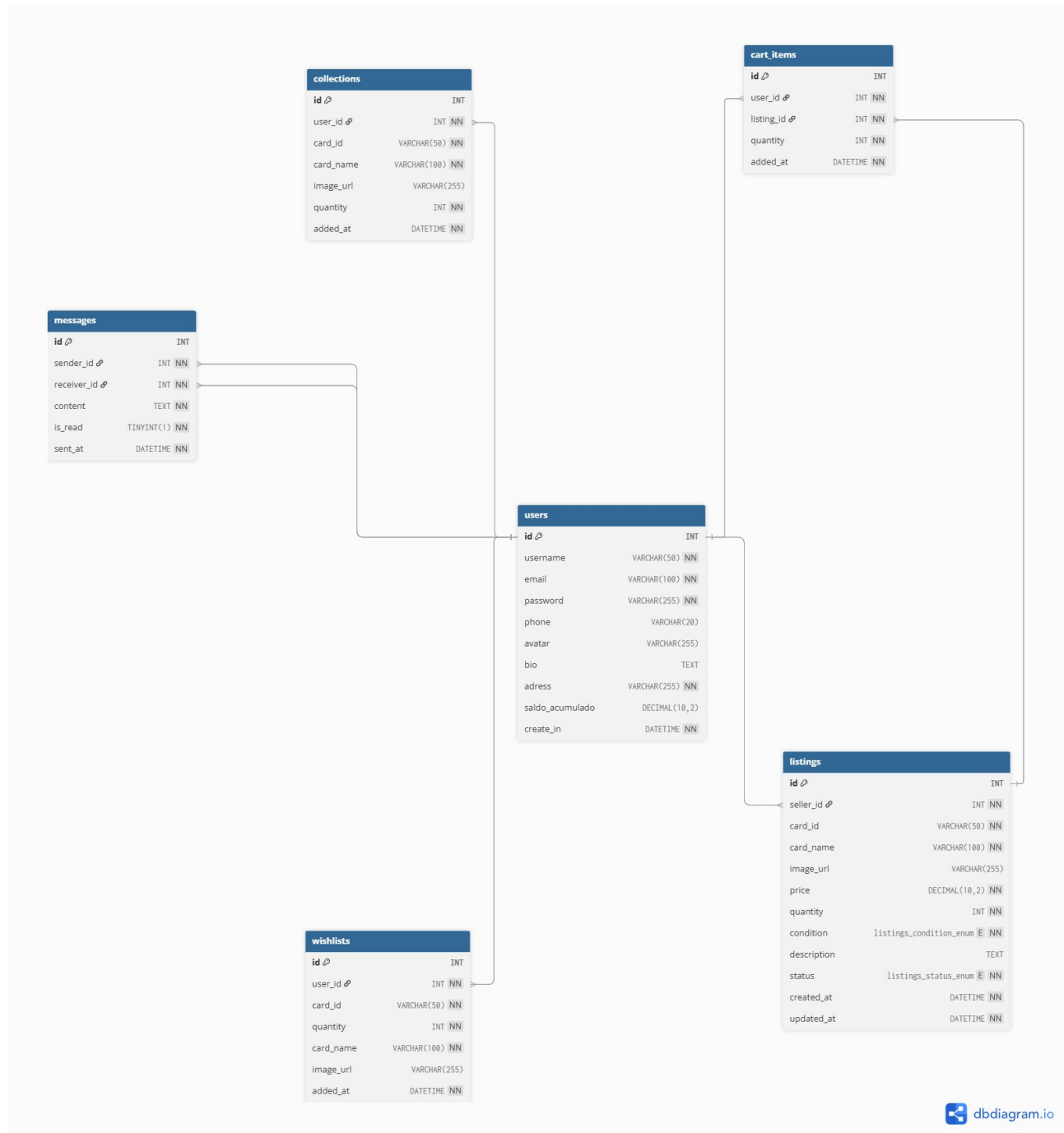


Figura 2: Diagrama entidad relación

Diseño de módulos/ficheros de código

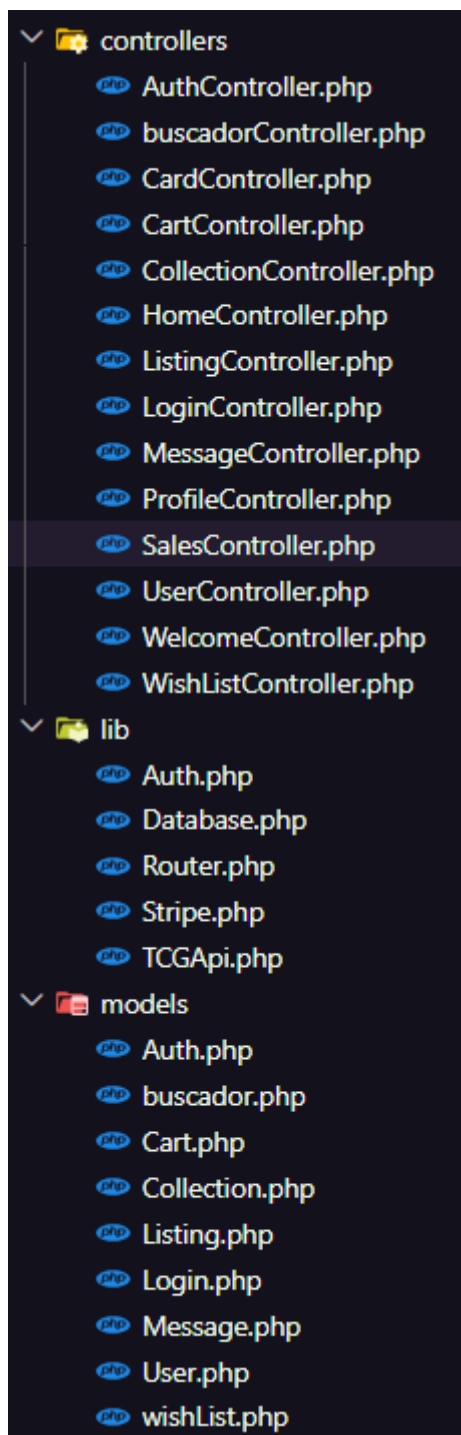


Figura 4: Diseño de ficheros

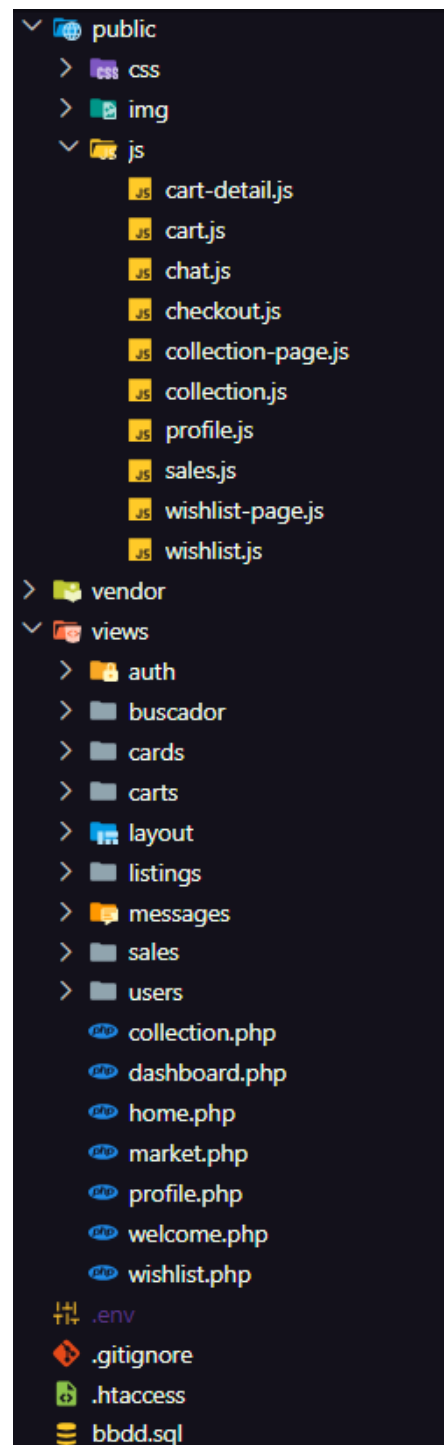


Figura 3: Diseño de ficheros

Diagrama de clases

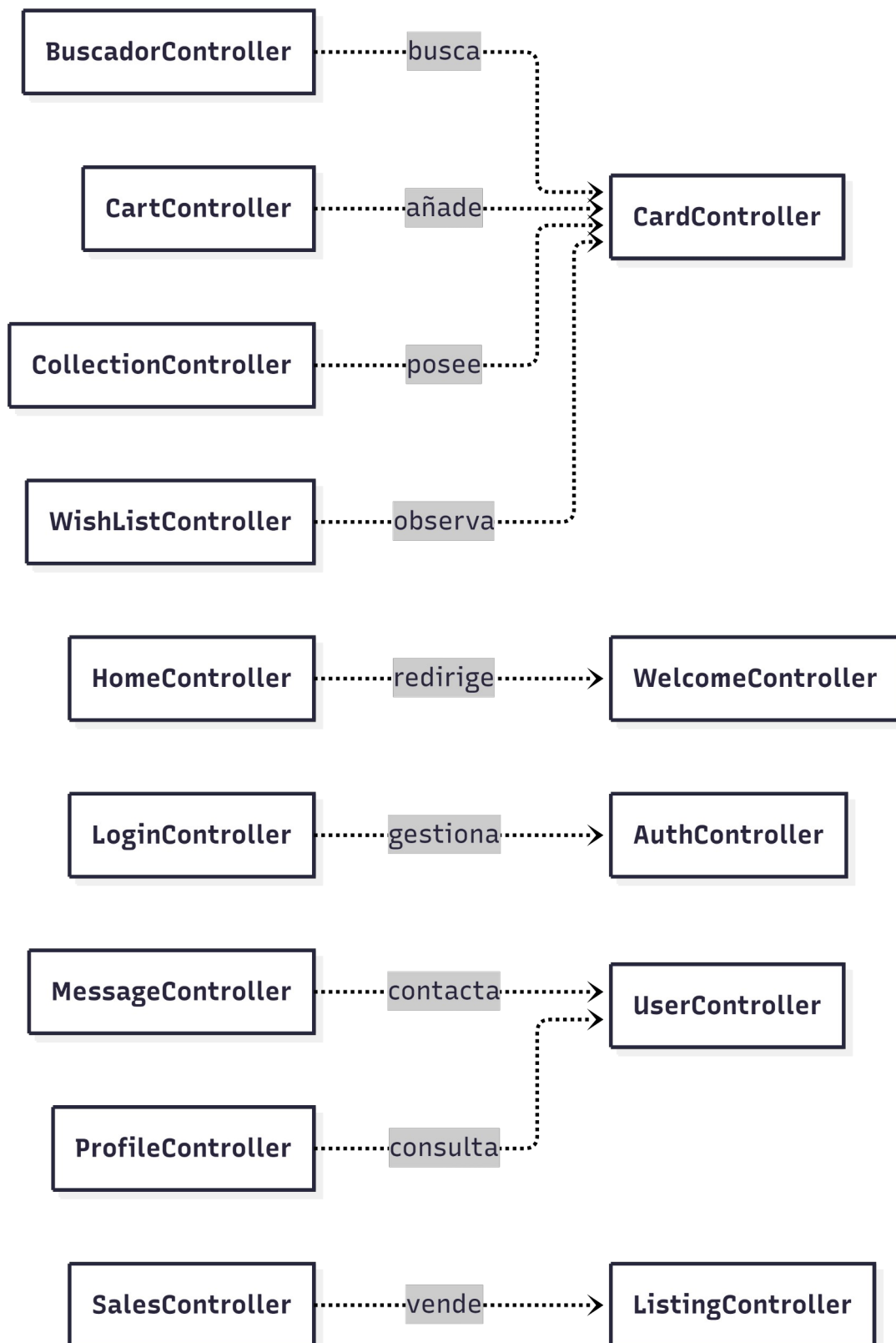


Figura 5: Diagrama de Clases

Diagrama de navegación

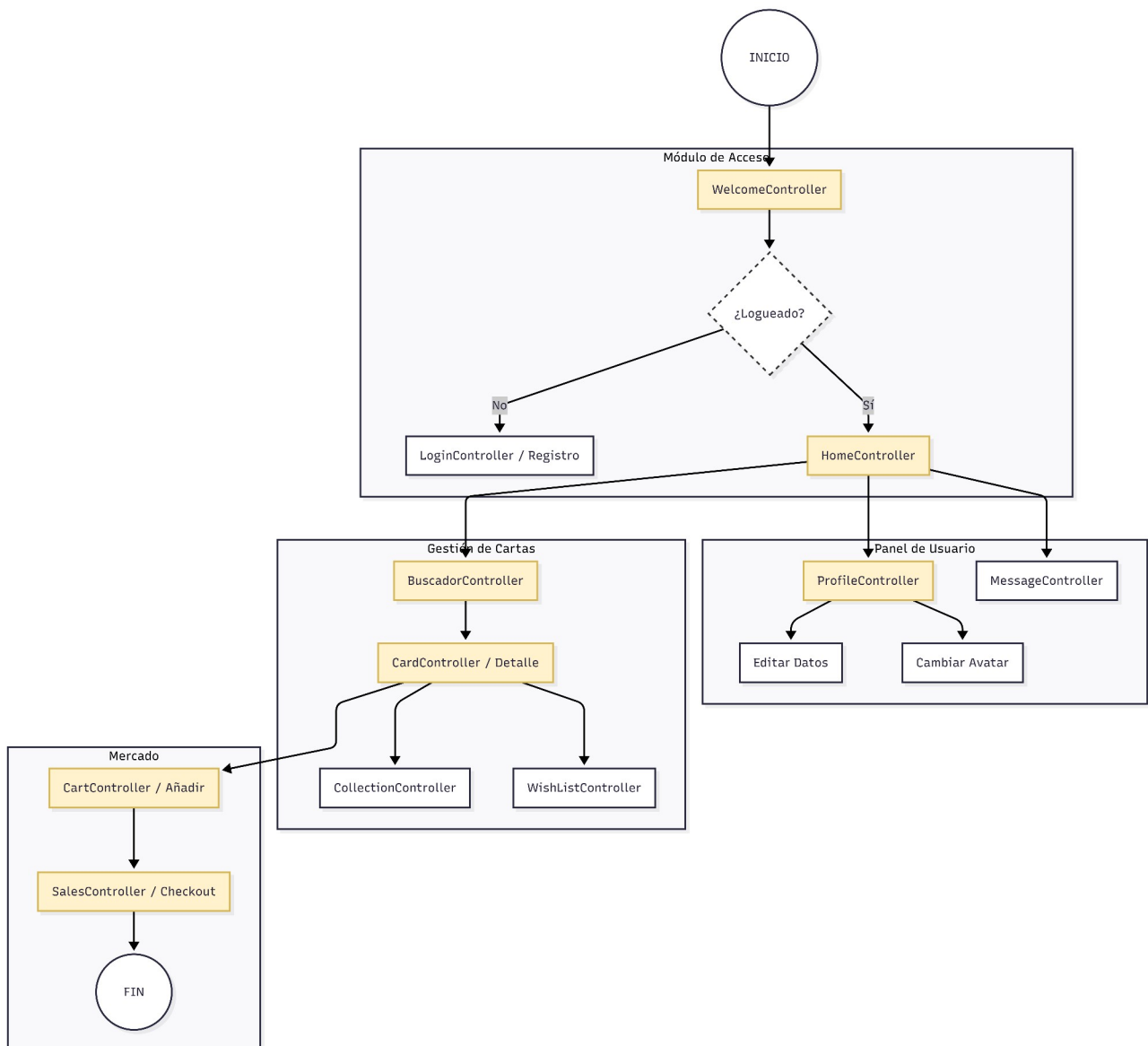


Figura 6: Diagrama de navegación

Trabajo de implementación

Introducción.

El proyecto desarrollado consiste en una aplicación web orientada a la gestión de compraventa de cartas coleccionables TCG. La plataforma permite a los usuarios registrarse, iniciar sesión, buscar cartas mediante integración con APIs externas, añadir productos al carrito, gestionar listas de deseos, administrar colecciones personales y realizar simulaciones de compra mediante una pasarela de pago.

La aplicación ha sido desarrollada siguiendo una arquitectura MVC (Modelo-Vista-Controlador), utilizando PHP como lenguaje principal en el backend y tecnologías web estándar en el frontend.

Además, el sistema incorpora autenticación de usuarios, validaciones, integración con Stripe para la pasarela de pagos y consumo de una API externa de cartas coleccionables.

Tecnologías y Herramientas

Para el desarrollo del proyecto se utilizaron las siguientes tecnologías y herramientas:

Backend

- PHP. Lenguaje principal utilizado para el desarrollo de la lógica del servidor.
- Arquitectura MVC. Organización del proyecto separando modelos, vistas y controladores.
- Composer. Gestor de dependencias utilizado para instalar librerías externas.
- MySQL. Sistema de gestión de bases de datos relacional.
- PHP Sessions. Utilizadas para la gestión de autenticación y persistencia de usuarios.

Librerías y APIs.

- Stripe PHP SDK. Librería utilizada para la simulación y gestión de pagos.
- TDGdex API. API externa utilizada para obtener información y cartas de coleccionables TCG.

Frontend

- HTML5. Estructuración de las páginas.
- CSS3. Diseño visual y estilos.
- JavaScript. Interacción dinámica en cliente.
- AJAX. Comunicación asíncrona entre clientes y servidor.

Herramientas de Desarrollo

- Git y GitHub. Control de versiones y almacenamiento del repositorio.
- XAMPP. Entorno local de desarrollo.
- PhpMyAdmin. Gestión visual de la base de datos.
- Visual Studio Code. Editor principal del project

Desarrollo

Backend

El backend de la aplicación se desarrolló utilizando PHP siguiendo una arquitectura MVC (Modelo-Vista-Controlador). Esta estructura permitió separar la lógica de negocio, el acceso a datos y la representación visual de la aplicación, facilitando el mantenimiento, reutilización y escalabilidad del proyecto.

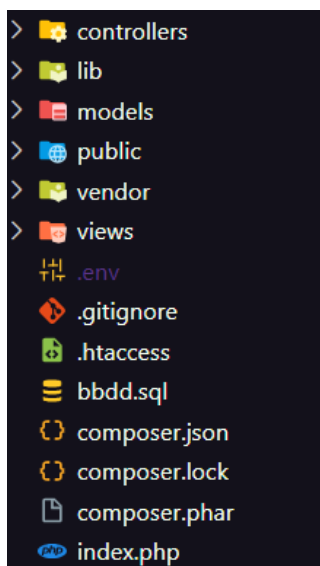


Figura 7: Estructura de carpetas siguiendo arquitectura MVC

La aplicación se organiza principalmente en:

- Modelos. Encargados de gestionar el acceso a la base de datos y la lógica relacionada con los datos.
- Vistas. Responsables de mostrar la interfaz al usuario.
- Controlador. Gestiona las peticiones HTTP y coordina la comunicación entre modelos y vistas.

Delia Méndez Navarro

Además, el proyecto incorpora un sistema de rutas personalizado mediante index.php, permitiendo gestionar dinámicamente las URLs y asociarlas con cada controlador y método correspondiente.

A continuación se muestra parte del sistema de rutas encargado de asociar URLs con controladores y métodos específicos.

```
// Inicio de Sesion
$router->add('/TFG/Codigo/login', 'LoginController@index');
$router->add('/TFG/Codigo/auth', 'LoginController@authenticate');

// Registro de usuario
$router->add('/TFG/Codigo/registro', 'LoginController@showRegister');
$router->add('/TFG/Codigo/do-register', 'LoginController@register');
```

Figura 8: Implementación del sistema de rutas personalizado.

Se implementaron las siguientes funcionalidades:

- Gestión y autenticación:

La aplicación permite el registro, inicio de sesión, cierre de sesión y eliminación de cuentas mediante los controladores LoginController y AuthController.

- Gestión de perfiles:

Los usuarios pueden modificar y visualizar su perfil desde ProfileController.

- Sistema de búsqueda:

El controlador buscadorController permite añadir productos permite realizar búsquedas dinámicas de cartas mediante peticiones al servidor.

- Sistema de carrito:

El controlador CartController permite añadir productos al carrito, eliminarlos y simular el proceso de compra.

- Gestión de cartas coleccionables TCG:

La aplicación integra información obtenida desde la API externa de TCGdex para mostrar cartas, colecciones e información detallada.

A continuación se muestra dos funciones que forman parte de TCGApi.php que ha hecho más fácil la obtención de datos de la API

```
private const BASE_URL = 'https://api.tcgdex.net/v2/en';

public static function getSets(): array {
    return self::fetch(self::BASE_URL . '/sets') ?? [];
}

public static function getSet(string $setId): ?array {
    return self::fetch(self::BASE_URL . "/sets/{$setId}");
}

public static function getCard(string $id): ?array {
    return self::fetch(self::BASE_URL . "/cards/{$id}");
}

public static function searchCards(string $query, int $limit = 40): array {
    $query = strtolower(trim($query));
    $sets = self::getSets();
    $results = [];

    foreach ($sets as $set) {
        $setData = self::getSet($set['id']);
        if (empty($setData['cards'])) continue;

        foreach ($setData['cards'] as $card) {
            // Solo cartas con imagen
            if (empty($card['image'])) continue;

            if (str_contains(strtolower($card['name']), $query)) {
                $results[] = $card;
            }
        }

        if (count($results) >= $limit) break 2;
    }

    return $results;
}
```

Figura 9: Funciones de TCGApi.php

Delia Méndez Navarro

- Lista de deseos:

Los usuarios pueden gestionar sus cartas favoritas mediante WishController.

- Gestión de colección:

La aplicación permite almacenar cartas en la colección personal del usuario utilizando CollectionController.

- Sistema de ventas:

Los usuarios pueden crear publicaciones y gestionar ventas desde ListingController y SalesController.

- Sistema de mensajería:

La aplicación incorpora mensajería entre usuarios mediante MessageController.

- Gestión de usuarios:

UserController permite visualizar información pública de usuarios y realizar búsquedas.

- Navegación general:

HomeController y WelcomeController gestionan las páginas principales y navegación general del sitio.

Modelos implementados

Los modelos desarrollados gestionan toda la lógica de acceso a datos y comunicación con MySQL. Cada modelo encapsula consultas SQL y operaciones relacionadas con su funcionalidad correspondiente.

Validaciones implementadas.

Se implementaron distintas validaciones tanto en backend como mediante peticiones AJAX.

- Inicio de sesión:

La aplicación valida si el nombre de usuario y la contraseña pertenecen a un usuario registrado antes de iniciar sesión.

- Acceso a rutas privadas:

Se comprueba si el usuario está autenticado antes de acceder a funcionalidades privadas como el carrito, ventas o colección.

- Añadir productos al carrito

La aplicación valida si existe sesión iniciada antes de permitir añadir cartas al carrito.

- Gestión de colección y favoritos:

Las operaciones AJAX valida en tiempo real si una carta ya pertenece a la colección o lista de deseos de usuario.

- Validación de mensaje:

La aplicación verifica que el usuario autenticado exista antes de permitir el envío de mensajes.

- Validación AJAX:

Los controladores `CollectionController` y `WishListController` incorporan métodos específicos para responder peticiones AJAX mediante respuestas JSON.

Frontend

El frontend se desarrolló utilizando tecnologías web estándar para crear una interfaz dinámica, intuitiva y responsive.

Se implementaron las siguientes tecnologías:

- HTML5. Utilizado para estructurar todas las vistas de la aplicación.
- CSS3. Empleado para el diseño visual y adaptación responsive.
- JavaScript. Utilizado para añadir dinamismo e interacción en tiempo real
- AJAX. Implementación para realizar operaciones asíncronas de recarga de páginas.

Características implementadas

- Interfaz responsive. La aplicación adapta automáticamente su diseño a distintos dispositivos y tamaños de pantalla.

- Gestión dinámica de cartas. La información de cartas obtenidas de la API se muestran dinámicamente al usuario.
- Actualización en tiempo real. Operaciones como añadir cartas al carrito, favoritos o colección se realiza mediante AJAX.
- Formulario interactivos. Se desarrollaron formularios para login, registro, edición de perfil, ventas y mensajería.
- Navegación dinámica. El sistema de rutas permite navegar entre distintas secciones de la aplicación manteniendo organizada la estructura MVC.

Validación frontend.

- Formularios completos. Se comprueba que todos los campos obligatorios estén correctamente rellenos antes de enviar la información.
- Gestión visual de errores. La aplicación muestra mensajes dinámicos cuando ocurre algún error durante formularios o peticiones AJAX.
- Validaciones dinámicas. Las operaciones de carrito, colección y favoritos validan en tiempo real el resultado de cada acción antes de actualizar la interfaz.

Justificación

La elección de las tecnologías y herramientas se basó en los siguientes criterios:

- Arquitectura estándar y accesibilidad. El uso de HTML 5 y CSS3 garantiza una estructura web semántica y una interfaz visualmente atractiva y adaptable. Al combinar esto con JavaScript, se logra una experiencia de usuario dinámica y fluida sin sobrecarga de frameworks pesados, permitiendo un control total sobre el DOM y la lógica del cliente.
- Robustez en el Procesamiento del Lado del Servidor: Se seleccionó PHP para el desarrollo del backend debido a su madurez y excelente integración con bases de datos relacionales. Su modelo de ejecución es ideal para gestionar la lógica de negocio, el control de sesiones de usuario y la comunicación con el servidor de forma eficiente y segura.
- Seguridad y Fiabilidad en las Transacciones: Para la gestión de pagos, se integró Stripe. Esta herramienta se eligió por ser el estándar de la industria en cuanto a cumplimiento de normativas de seguridad (PCI DSS), permitiendo procesar pagos

de forma cifrada y segura sin que los datos sensibles de las tarjetas toquen directamente nuestro servidor.

- Integración de Datos Especializados: El uso de la API de TCGdex fue fundamental para dotar al proyecto de una base de datos de cartas coleccionables completa y actualizada. Esta herramienta permite consumir información técnica e imágenes de alta calidad de forma externa, optimizando el almacenamiento local y asegurando que la información de las cartas sea veraz y oficial.
- Facilidad de Desarrollo y Despliegue: El stack elegido (entorno LAMP/WAMP) destaca por su facilidad de configuración y su amplia comunidad de soporte. PHP y JavaScript permiten un ciclo de desarrollo ágil, facilitando la depuración y la implementación de nuevas funcionalidades de forma incremental.
- Modularidad y Limpieza de Código: Se ha priorizado la separación de responsabilidades siguiendo el patrón MVC (Modelo-Vista-Controlador). El uso de controladores en PHP para la lógica y archivos JS/CSS independientes para el comportamiento y estilo, asegura que el proyecto sea escalable y fácil de mantener a largo plazo.

Trabajo de pruebas.

Para las pruebas que devuelven los HTML y los flujos de usuario, se realizaron pruebas directamente desde el navegador utilizando las herramientas de desarrollo, prestando especial atención a la pestaña Network para verificar peticiones AJAX y sus respuestas.

- Flujo de registro y login. Se verificó que el sistema impide el registro con nombres de usuario o emails ya existentes, mostrando el error correspondiente en el formulario. Se comprobó también que las credenciales incorrectas no permiten el acceso.



The screenshot shows a login interface with the title "Iniciar Sesión" in red. Below the title is a pink error message box that says "Usuario o contraseña incorrectos.". There are two input fields: "Usuario" with a light blue border and the text "usuario no registrado" inside, and "Contraseña" with a red border and masked characters "....". Below these fields is a red button labeled "Entrar". At the bottom, there is a link that says "¿No tienes cuenta? **Regístrate aquí**".

Figura 10: Prueba de usuario no registrado

- Flujo de compra completo. Se realizaron compras de prueba con tarjetas test proporcionadas por Stripe (4242 4242 4242 4242, para pago exitoso y 4000 0000 0000 0002, para pago rechazado), verificando que el stock se actualiza correctamente, el carrito se vacía, el saldo del vendedor se actualiza y llega el mensaje con los datos del comprador y la compra.

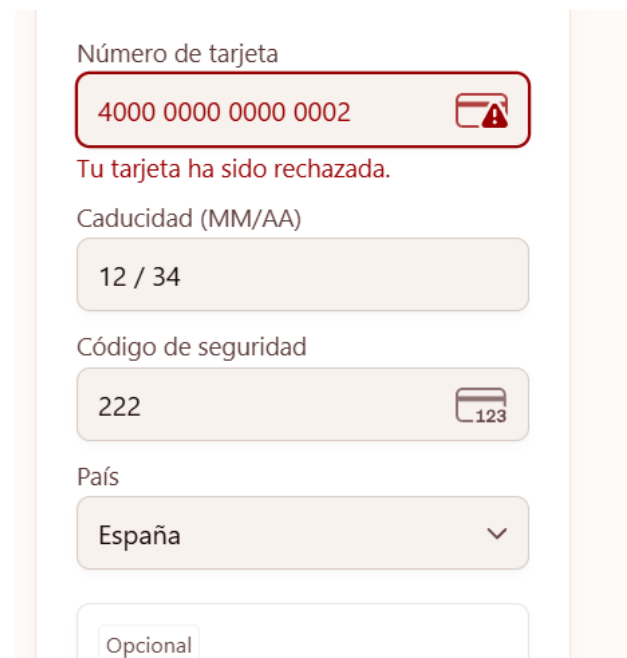
El formulario de pago está dividido en secciones. La primera sección, 'Número de tarjeta', muestra el número '4000 0000 0000 0002' en rojo con un icono de tarjeta y una advertencia. Debajo, un mensaje rojo indica 'Tu tarjeta ha sido rechazada.'. La segunda sección, 'Caducidad (MM/AA)', muestra '12 / 34'. La tercera, 'Código de seguridad', muestra '222' con un icono de tarjeta y el número '123'. La cuarta, 'País', muestra 'España' con una flecha hacia abajo. En la parte inferior, hay un campo 'Opcional' vacío.

Figura 11: Prueba tarjeta rechazada

- Gestión del perfil. Se comprobó la actualización de los datos personales, cambio de contraseña y la subida y eliminación del avatar.
- Mensajería. Se verificó el envío y recepción de mensajes entre dos cuentas distintas.



Figura 12: Mensajería funcional

- Validación de stock. Se comprobó que no es posible añadir al carrito mas unidades de las disponibles en el listing, y que los listings propios no pueden ser añadidos al carrito por el mismo vendedor.
- Protección de rutas privadas. Se intentó acceder directamente a rutas como /carrito, /perfil o /ventas sin sesión activa, confirmando que el sistema redirige al login en todos los casos.

Trabajo de implantación.

La fase de implantación describe el proceso de puesta en marcha de la aplicación en un entorno de producción, garantizando su disponibilidad y correcto funcionamiento.

Entorno de desarrollo y Servidor local.

Para el desarrollo y las pruebas iniciales, se ha utilizado un entorno WAMP (Windows, Apache, MySQL, PHP), que proporciona las herramientas necesarias para la ejecución del stack tecnológico:

- Servidor Web (Apache): Encargado de procesar las peticiones HTTP y servir los activos de la aplicación (PHP, JS, HTML y CSS).
- Base de Datos (MySQL): Utilizada para la persistencia de datos críticos como perfiles de usuario, inventario de cartas y registro de transacciones.
- Gestión de Dependencias: Se ha realizado una integración manual de librerías, destacando la SDK de Stripe para la gestión de la pasarela de pagos y la conexión mediante peticiones asíncronas a la API de TCGdex.

Configuración de servicios externos.

La funcionalidad de la plataforma depende de la correcta vinculación con servicios de terceros, lo que requiere una configuración específica de seguridad y conectividad:

- Pasarela de pagos (Stripe). Se ha implementado una integración directa utilizando las claves de API (Publishable y Secret Key). Para garantizar la seguridad de la plataforma, estas credenciales no se encuentran expuestas en el código fuente, sino que se gestionan a través de un archivo de configuración de entorno .env, accesible únicamente por el servidor. El flujo de pago se completa con una redirección controlada que verifica el estado de la transacción antes de confirmar el pedido al usuario.
- Fuente de Datos (TCGdex): La aplicación consume información técnica e imágenes en tiempo real a través de esta API externa. Esto optimiza el rendimiento del servidor local al no tener que almacenar miles de archivos de imagen, asegurando

que el catálogo de cartas coleccionables esté siempre actualizado con datos oficiales.

Arquitectura de despliegue y rutas.

A diferencia de un sitio web estático, esta aplicación requiere una configuración específica del servidor para soportar el patrón MVC (Modelo-Vista-Controlador):

- Puerta de Enlace Única (index.php): Se ha configurado el servidor Apache (mediante un archivo .htaccess) para que todas las peticiones sean redirigidas al archivo raíz. Esto permite que el Router personalizado procese las URLs amigables (ej. /perfil o /carrito) y las asigne al controlador adecuado.
- Seguridad de Directorios: La estructura de carpetas protege los archivos sensibles (como modelos y controladores), impidiendo el acceso directo desde el navegador y forzando a que toda interacción pase por la lógica de seguridad establecida.

Proceso de puesta en producción.

Para el despliegue final en un entorno real, se definieron los siguientes pasos técnicos:

- Migración de Base de Datos. Exportación del esquema SQL local e importación en el servidor de producción.
- Configuración del Entorno Seguro. Ajuste del archivo .env con las credenciales de producción y activación obligatoria del certificado SSL (HTTPS), requisito técnico indispensable para que la pasarela de Stripe procese pagos de forma cifrada.
- Sincronización de Rutas. Actualización de la URL base del proyecto para garantizar la correcta carga de activos (JS/CSS) bajo el nuevo dominio.

Guía de instalación y ejecución local

Para replicar el entorno de ejecución en un equipo local, se deben seguir estos pasos.

- Instalación del servidor. Descargar e instalar XAMPP que incluya soporte para PHP 8.0 o mas y MySQL.
- Despliegue del código. Copiar la carpeta del proyecto dentro del directorio raíz del servidor, ubicado habitualmente en <C:/xampp/htdocs/>
- Inicialización de la base de datos. Acceder a phpMyAdmin, crear una nueva base de datos e importar el archivo .sql incluido en la carpeta del proyecto para generar las tablas y los datos iniciales.
- Configuración del entorno. Asegurar que el archivo .env contenga las credenciales correctas de acceso a la base de datos local.
- Ejecución. Iniciar los módulos de Apache y MySQL desde el panel de control de XAMPP y acceder a la aplicación a través del navegador mediante la dirección: localhost/TFG/Codigo.

Producto

Descripción externa.

Página de inicio.

La página de inicio tiene una pequeña descripción de lo que es la web, dando acceso al inicio de sesión y la pagina de registro.



Figura 13: Página de inicio

Nuevo registro.

Este formulario facilita el proceso de registro para nuevos usuarios. Incluye validaciones en el nombre y mail para que no haya dos usuarios con los mismos datos lo que ayuda a prevenir duplicados en la base de datos.

The image shows a registration form titled 'Crear cuenta' on a light beige background. The form is a white card with the following fields: 'Usuario' (with placeholder 'Tu nombre de usuario'), 'Email' (with placeholder 'correo@ejemplo.com'), 'Contraseña' (with masked characters), 'Teléfono' (with placeholder 'Ej: 600000000'), and 'Dirección' (with placeholder 'Calle inventada, Ficticio, 20'). At the bottom of the form is a red button labeled 'Registrarse'. Below the button, there is a link that says '¿Ya tienes cuenta? Inicia sesión'.

Figura 14: Registro

Delia Méndez Navarro

Login.

La interfaz permite al usuario iniciar sesión una vez registrado.

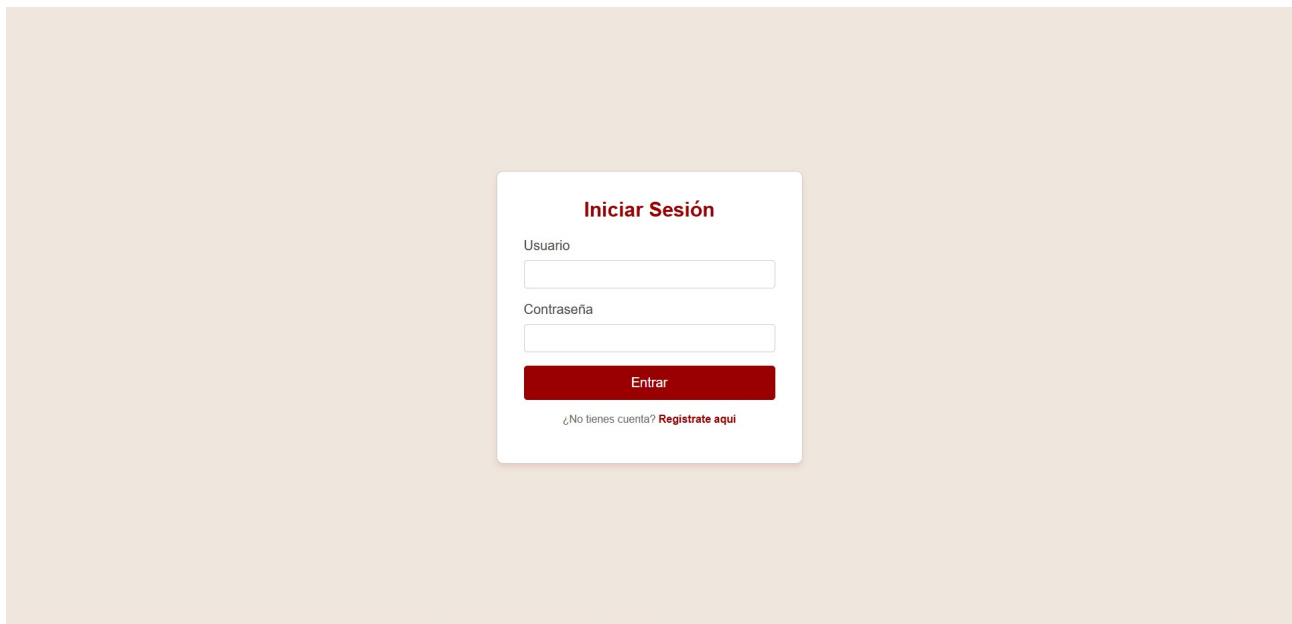


Figura 15: Página de Inicio de sesión

Usuario con sesión iniciada

Después de iniciar sesión el usuario es redirigido al home donde muestra un mensaje personalizado con el nombre de usuario, las cartas destacadas en ese momento, y acceso al menú.

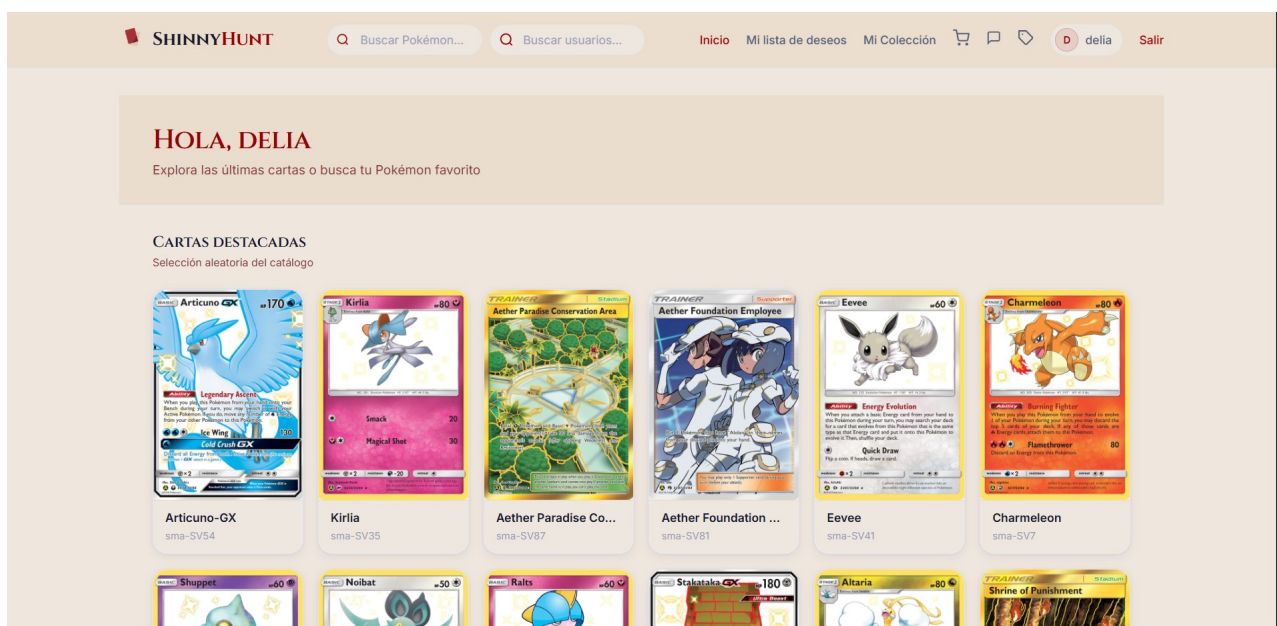


Figura 16: Página principal

Delia Méndez Navarro

Detalle de carta

Al pulsar una carta el usuario es redirigido a los detalles de esta mostrando nombre, set, precio, ilustrador y contenidos importantes de la carta como el ataque/acción, puntos de vida, coste de retirada, etc. Más abajo se encuentran los botones para añadirla a la lista de deseos, colección o puesta en venta.

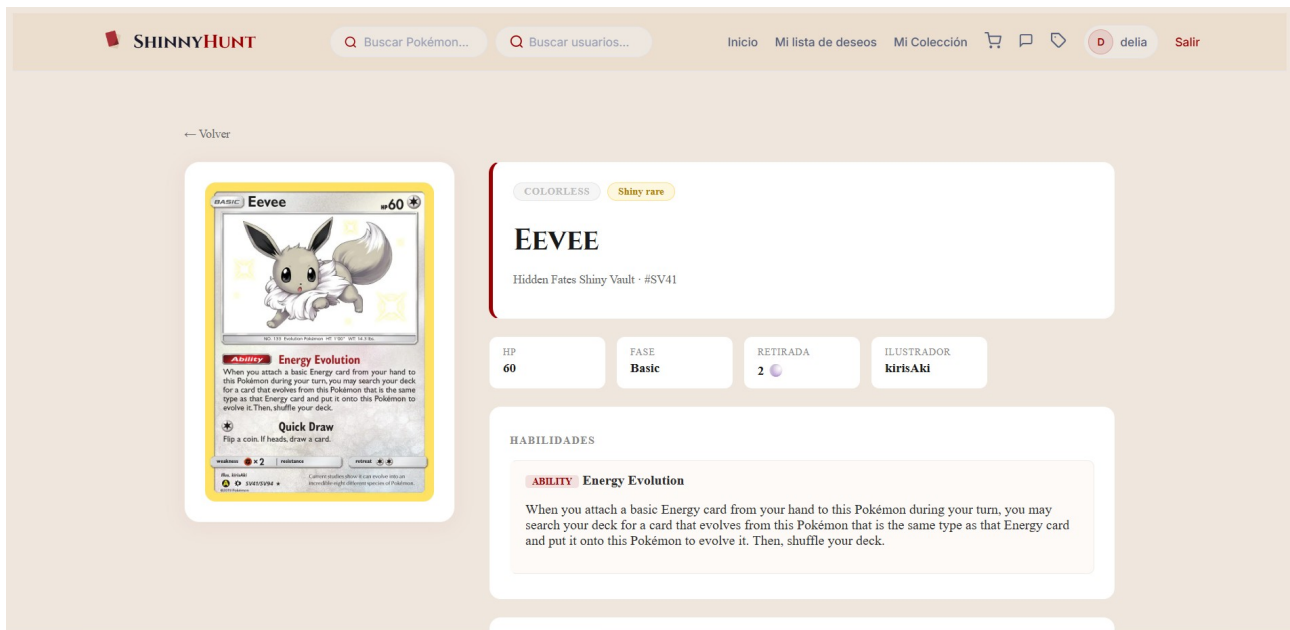


Figura 17: Detalle de la carta

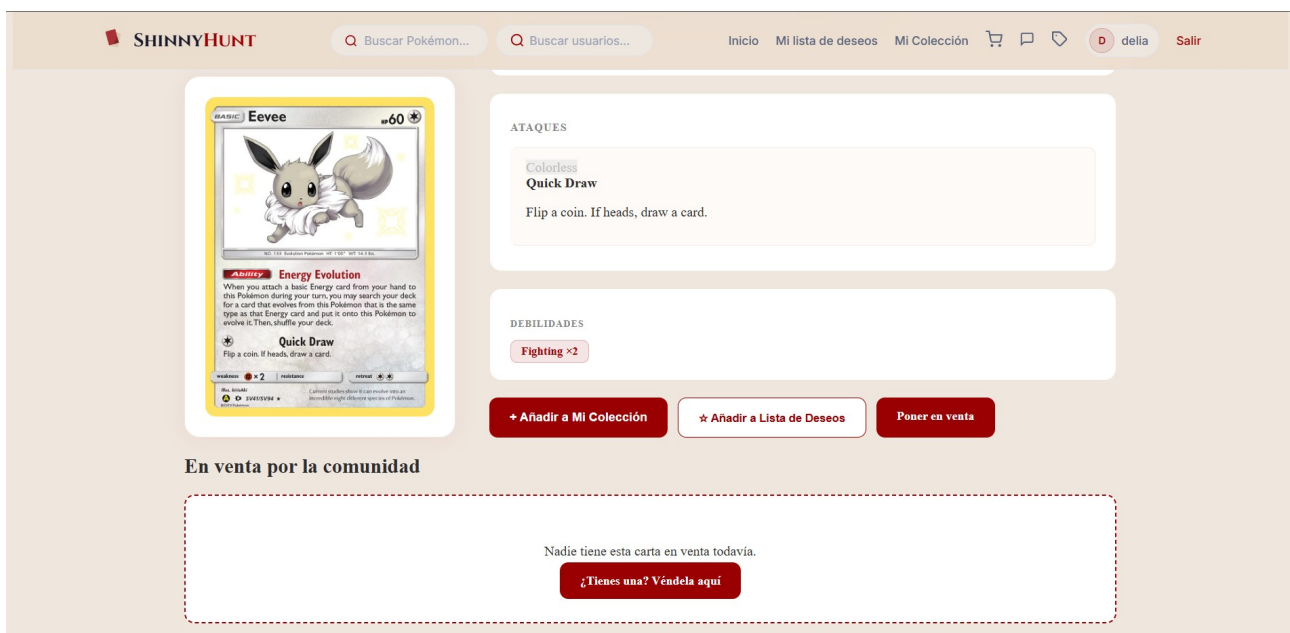


Figura 18: Detalle de la carta

Formulario de venta

En el formulario de venta el usuario debe rellenarlo con precio, cantidad, estado de la carta y una descripción opcional para que posteriormente se muestre en su listing.



Figura 19: Formulario de venta

Buscador de carta

Al buscar una carta se muestran las 40 primeras cartas coincidentes, para ver las 40 siguientes en caso de que haya, en la parte inferior de la búsqueda hay un botón “ver siguientes 40”.

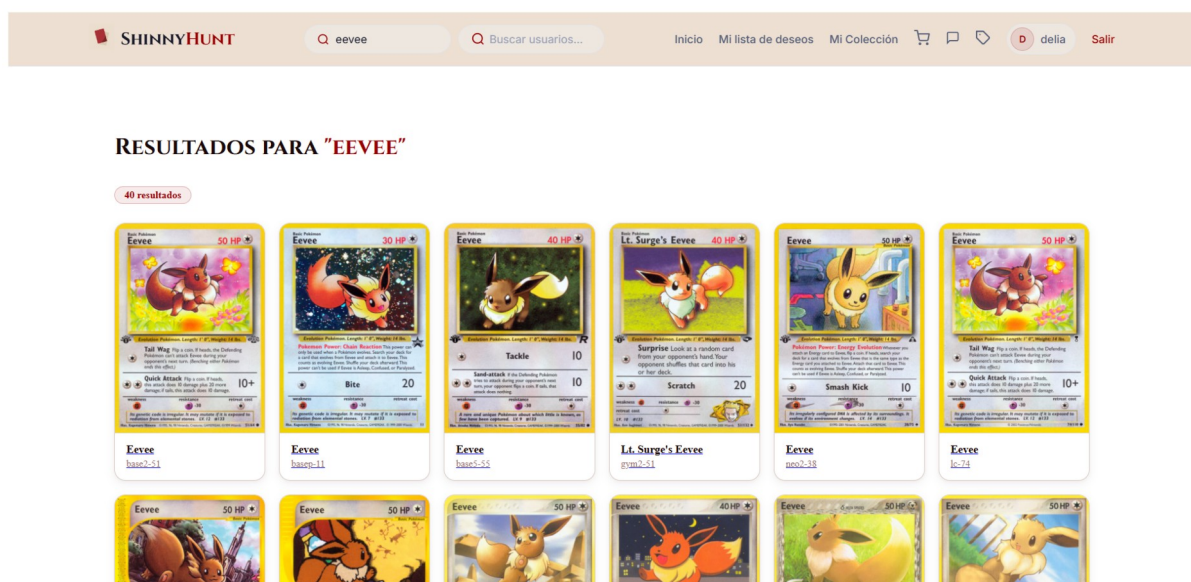


Figura 20: Buscador de carta

Delia Méndez Navarro

Buscador de usuario

En la búsqueda de usuarios se muestran los usuarios con ese nombre.



Figura 21: Buscador de usuario

Perfil de usuario ajeno

Al pulsar en el nombre de un usuario te muestra su perfil público donde el usuario puede ver las cartas de este en venta y tiene un acceso a un chat con el usuario.

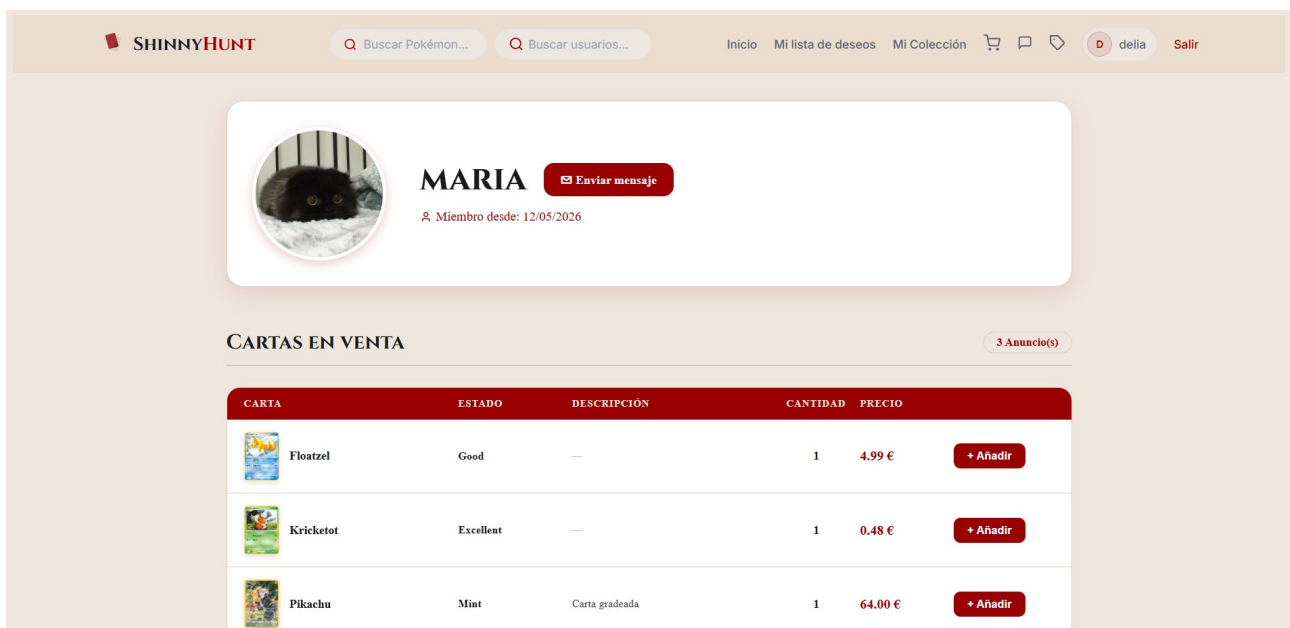


Figura 22: Perfil de usuario

Colección

Un usuario puede acceder a su colección virtual y ver todas las cartas que tiene en ella a parte de poder añadir nuevas pulsando el botón explorar.

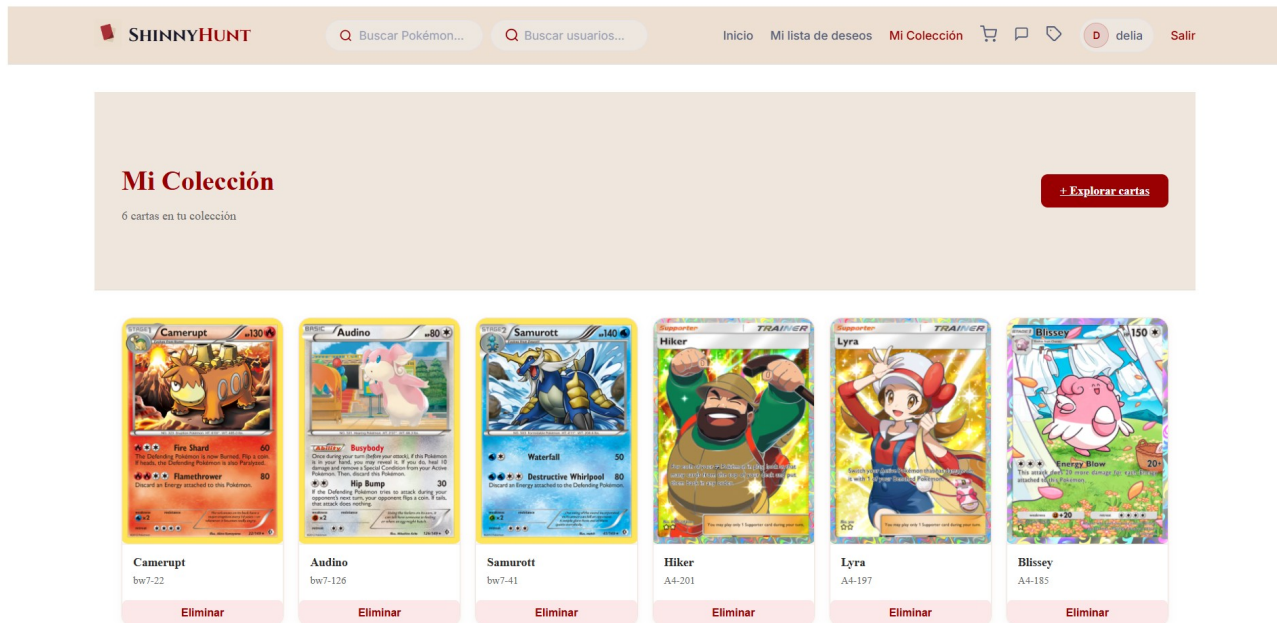


Figura 23: Vista colección

Lista de deseos

Un usuario puede acceder a su lista de deseos virtual y ver todas las cartas que tiene en ella a parte de poder añadir nuevas pulsando el botón explorar.

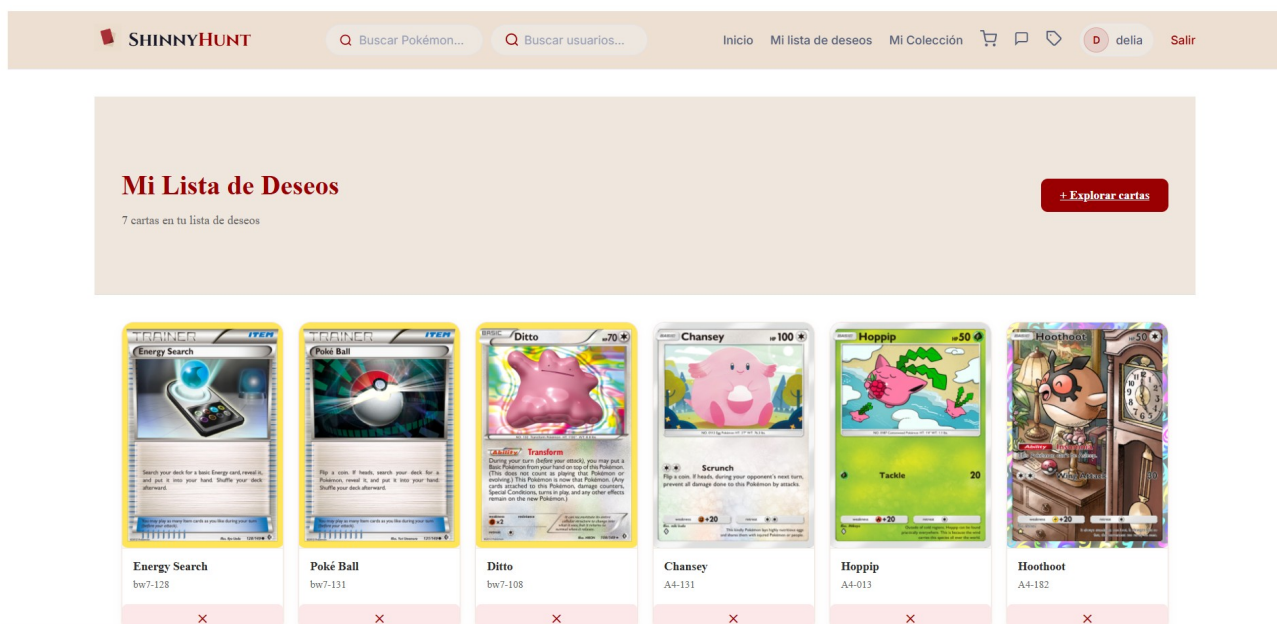


Figura 24: Vista lista de deseos

Mensajes

EL usuario puede mandar y recibir mensajes de otros usuarios, y en caso de que un usuario compre una carta del usuario llegará un mensaje con los datos.

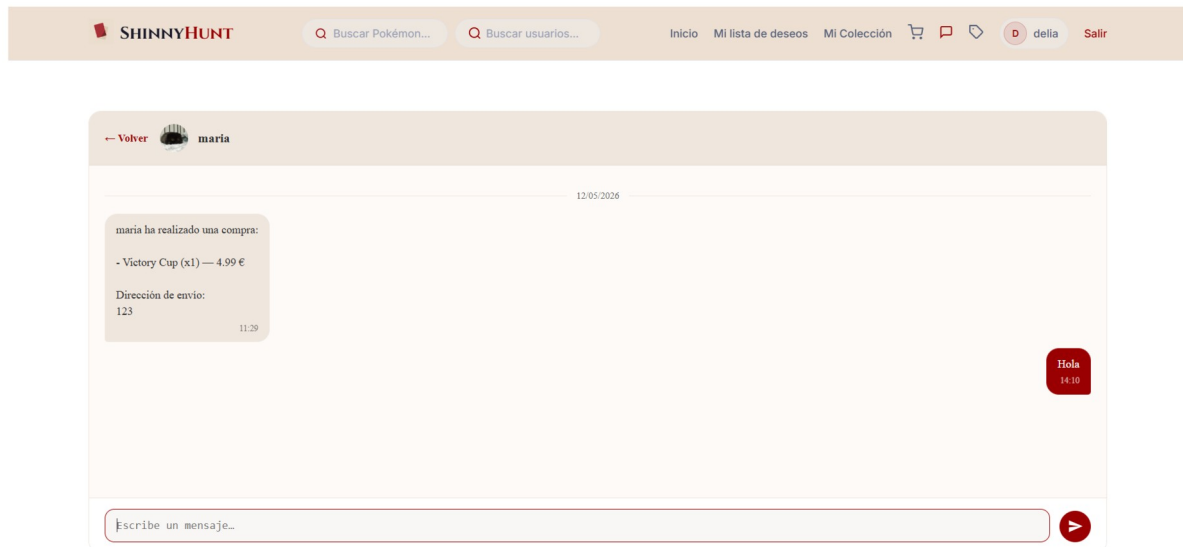


Figura 25: Mensajería

Carrito

En el carrito saldrán las cartas que el usuario a añadido al carrito y a la derecha un botón que desplegará una pasarela de pago para realizar el pago.

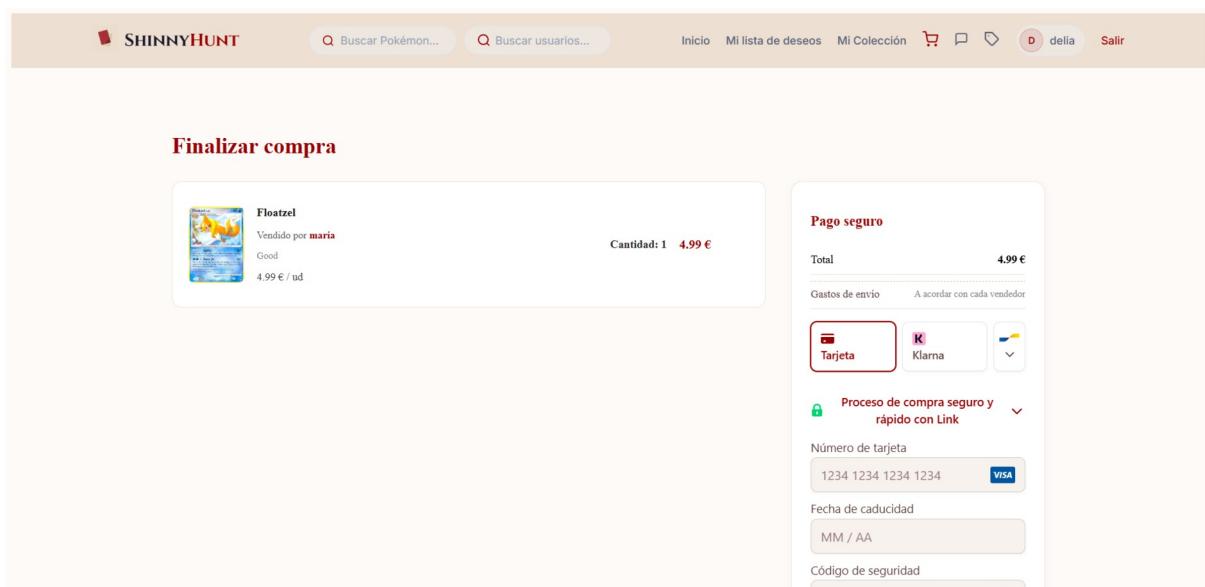


Figura 26: Carrito

Delia Méndez Navarro

Perfil de usuario

En este apartado el usuario puede consultar sus datos y cambiarlos. Puede añadir una descripción, cambiar el nombre de usuario, mail, numero de teléfono, dirección y contraseña. En caso de querer darse de baja puede eliminar la cuenta. También puede ver el dinero que ha obtenido gracias a sus ventas,

SHINNYHUNT Inicio Mi lista de deseos Mi Colección **delia** Salir



delia
delia@example.com

6
EN COLECCIÓN

7
EN WISHLIST

4.99 €
GANANCIAS DE TUS VENTAS
Retirar saldo

Datos personales

Nombre de usuario

delia

Email

delia@example.com

Teléfono

123

Biografía

Cuéntanos algo sobre ti...

Dirección

Direccion inventada

Guardar cambios

Cambiar contraseña

Contraseña actual

Nueva contraseña

Confirmar nueva contraseña

Cambiar contraseña

Zona Peligrosa

Una vez que elimines tu cuenta, no hay marcha atrás. Por favor, asegúrate.

Eliminar mi cuenta permanentemente

Figura 27: Perfil de usuario

Venta

En este apartado se muestran las cartas puestas en venta por el usuario, también se mostrarán las cartas vendidas en caso de querer consultarlo en algún momento.

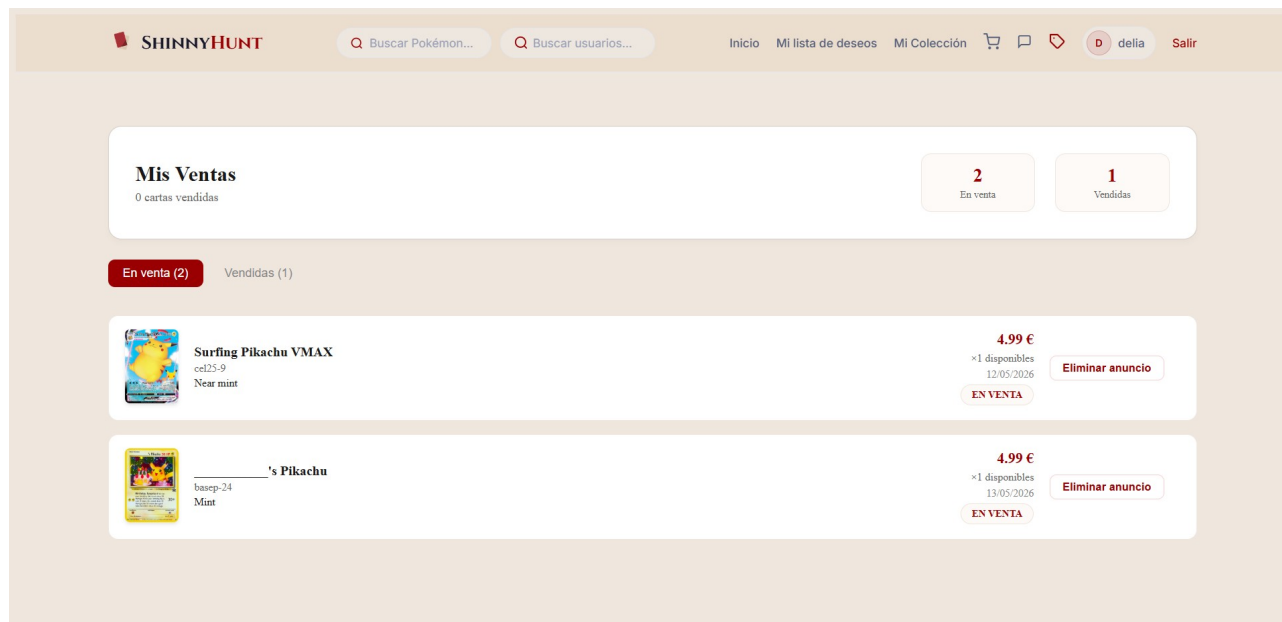


Figura 28: Vista de venta

Infraestructura.

La infraestructura del proyecto comprende un conjunto de herramientas y servicios que permiten que la aplicación funcione de manera estable y segura. Se ha diseñado priorizando la eficiencia en el manejo de datos y la protección de la información sensible.

Entorno de ejecución local.

Para que la aplicación funcione durante su fase de desarrollo y presentación, se ha utilizado el paquete XAMPP, que integra los tres pilares principales de la infraestructura:

- Servidor Apache. Componente encargado de gestionar las peticiones a los usuarios y entregar los archivos de la web.
- Interprete de PHP. El motor que procesa la lógica de la aplicación y conecta la base de datos con la interfaz visual.
- Sistema MySQL. Donde se organiza y guarda toda la información de forma permanente toda la información relativa a los usuarios y productos.

Gestión de la Seguridad y Configuración

Uno de los puntos clave de la infraestructura es el aislamiento de las credenciales críticas. Se ha implementado un sistema basado en variables de entorno (.env):

- Esta técnica permite que las claves privadas de la base de datos y de la pasarela de pagos no estén escritas directamente en el código.
- Funciona como una capa de seguridad adicional que impide que información confidencial sea visible si el código se traslada a otros entornos o repositorios.

Integración de Servicios en la Nube (APIs)

La infraestructura no es un sistema cerrado, sino que se apoya en servicios externos especializados para ofrecer funcionalidades avanzadas:

- Servicios de Pago (Stripe): En lugar de gestionar datos bancarios en el servidor propio, la infraestructura delega el cobro a Stripe. Esto garantiza que el proceso de pago cumpla con los estándares de seguridad bancaria más estrictos sin aumentar la complejidad de nuestra base de datos.
- Suministro de Información (TCGdex): El catálogo de cartas se alimenta de una infraestructura externa. Esto permite que la aplicación sea mucho más ligera, ya que las imágenes y los detalles técnicos de las cartas se cargan directamente desde sus servidores, ahorrando espacio de almacenamiento local.

Organización de Rutas y Navegación

Para que la infraestructura sea profesional, se ha configurado el servidor mediante un archivo .htaccess. Este componente permite que las direcciones web sean limpias y fáciles de entender para el usuario (por ejemplo, transformando rutas complejas en nombres sencillos como /login o /tienda), mejorando así la organización interna del sitio.

Conclusión.

Resumen del proyecto.

El presente proyecto ha cumplido el objetivo principal: el desarrollo de una plataforma E-commerce funcional y segura para el coleccionismo de cartas TCG. Se ha logrado integrar con éxito una arquitectura MVC sólida en PHP, un sistema de pagos mediante Stripe y un catálogo dinámico alimentado por la API de TCGdex. La aplicación no solo permite la compra-venta, sino que garantiza una experiencia de usuario fluida y una gestión de datos protegida.

Lecciones aprendidas.

- La importancia de no exponer datos sensibles en el código, y el uso de archivos .env para proteger claves privadas como pueden ser las de Stripe, utilizadas en este proyecto.
- Interpretar documentaciones técnicas complejas y el manejo de comunicación asíncrona con APIs de terceros como Stripe y TCGdex, adaptando sus servicios a las necesidades del proyecto.
- Identificación de errores mediante Postman y el apartado de Network del propio navegador para optimizar el backend y mejorar la comunicación entre cliente y servidor.

Posible evolución futura.

- Mejorar las funcionalidades ya existentes, adaptar el sistema de mensajería a un sistema de mensajería instantáneo.
- Implementar traducciones en otros idiomas para llegar a un público más amplio.
- Añadir un sistema de alertas, vía mail o avisos web, para avisar a los usuarios cuando un producto de su lista de deseos baje de precio.
- Integrar una inteligencia artificial que ayude a los vendedores a tasar sus cartas basándose en el estado de conservación y precio medio del mercado actual.

Bibliografía

Herramientas de diseño y planificación.

- Instagantt. (s.f.). *Software de gestión de proyectos y diagramas de Gantt*. Recuperado de <https://www.instagantt.com>
- Mermaid AI (s.f.). *Herramienta de diagramación basada en texto*. Recuperado de <https://mermaid.ai>
- Notion (s.f.). *Plataforma de organización y gestión documental*. Recuperado de <https://www.notion.so>

Servicios de terceros y APIs

- Stripe (s.f.). *Accept a payment (Payment Intents API) for Web*. Recuperado de <https://stripe.com/docs/payments/accept-a-payment>
- Stripe. (s.f.). *Security and PCI Compliance for Online Payments*. Recuperado de <https://stripe.com/docs/security/guide>
- Stripe. (s.f.). *Stripe API Reference for PHP SDK*. Recuperado de <https://stripe.com/docs/api?lang=php>
- Stripe. (s.f.). *Testing your Stripe Integration*. Recuperado de <https://stripe.com/docs/testing>
- Stripe Samples. (s.f.). *Checkout One-Time Payments with PHP and JavaScript* [Código fuente]. GitHub. <https://github.com/stripe-samples/checkout-one-time-payments>
- Cloudways. (s.f.). *Integrating Stripe Payment Gateway in PHP: A Step-by-Step Guide*. Recuperado de <https://www.cloudways.com/blog/stripe-payment-gateway-php/>
- The PHP Foundation. (s.f.). *Stripe-PHP: Official Stripe PHP client library*. Recuperado de <https://github.com/stripe/stripe-php>

Documentación de tecnologías y APIs

- PHP Group. (s.f.). *Manual de PHP (Versión 8.x)*. Recuperado de <https://www.php.net/manual/es/>
- TCGdex. (s.f.). *TCGdex API Documentation*. Recuperado de <https://tcgdex.dev/>

Delia Méndez Navarro

- W3Schools. (s.f.). *PHP MySQL Database Integration*. Recuperado de <https://www.w3schools.com/php/>